

Python Tip: Lambda Functions

Need a quick function for a single evaluation? Use `lambda` functions to write compact, one-line functions without defining a whole block of code.

What is a Lambda Function?

A lambda function is exactly like a regular function, but it's defined in a single line and can only contain one expression (no if statements, loops, etc.).

It's often used for simple tasks where you don't need a full function definition.

```
# Basic syntax: lambda arguments: expression
add = lambda x, y: x + y

print(add(3, 4))  # 7
```

7

Lambda functions are useful when you need to perform a quick evaluation without writing a full function definition.

Why Use Lambda Functions?

Lambda functions let you perform simple operations without a full function declaration. This makes your code shorter and easier to read.

```
# Example: Sorting a list of tuples based on the second element
data = [(1, 2), (3, 1), (5, 3)]

# Use lambda to define a function to sort based on the second element
data.sort(key=lambda x: x[1])

print(data)  # [(3, 1), (1, 2), (5, 3)]
```

```
[(3, 1), (1, 2), (5, 3)]
```

This is especially useful when you only need a function temporarily and don't want to clutter your code.

Using Lambda with `map()` and `filter()`

Lambda functions are commonly used with `map()` and `filter()` where you need to apply a simple operation to each element in a sequence.

```
numbers = [1, 2, 3, 4, 5]

# Using lambda with map to square each number
squared = map(lambda x: x**2, numbers)
print(list(squared))  # [1, 4, 9, 16, 25]
```

```
[1, 4, 9, 16, 25]
```

```
numbers = [1, 2, 3, 4, 5]

# Using lambda with filter to keep only even numbers
even_numbers = filter(lambda x: x % 2 == 0, numbers)
print(list(even_numbers))  #
```

Lambda Functions in Other Languages

Lambda functions are also known as “anonymous functions” because they don’t have a name like regular functions. Instead, they’re defined inline for short, one-off tasks.

Many other languages support anonymous functions: -

JavaScript: `const add = (x, y) => x + y;` - **Julia:** `add = (x, y) -> x + y` - **Ruby:** `add = ->(x, y) { x + y }`

Wrap-Up

Now you can use lambda functions for quick, one-line functions to keep your code short and simple.

Follow me for more tips.

Shep Bryan IV